2026-9-29 Linux五种IO模型及其工作原理与应用(完整版）

- 1. IO 是什么？
  - 1.1 IO 的本质：等 + 拷贝
  - 1.2 IO 慢的根本原因：等的时间太多
  - 1.3 什么叫做高效的 IO？
- 2. 讲一个故事，直观理解 IO
  - 2.1 背景阐述
  - 2.2 五种钓鱼方式
    - 2.2.1 张三 —— 阻塞 IO
    - 2.2.2 李四 —— 非阻塞 IO
    - 2.2.3 王五 —— 信号驱动 IO
    - 2.2.4 赵六 —— 多路复用 / 多路转接
    - 2.2.5 田七 —— 异步 IO
  - 2.3 五种钓鱼方式的效率对比：赵六为什么最快？
- 3. 五种 IO 模型详解
  - 3.1 阻塞 IO
    - 3.1.1 概念：最常见、最简单、最可靠
    - 3.1.2 工作过程：等 + 拷都在内核完成
  - 3.2 非阻塞 IO
    - 3.2.1 概念：等的时候不会卡住
    - 3.2.2 工作过程：没就绪立即返回
  - 3.3 信号驱动 IO
    - 3.3.1 概念：通过 SIGIO 信号通知
    - 3.3.2 工作过程：绑定 SIGIO + O_ASYNC
  - 3.4 多路复用 / 多路转接
    - 3.4.1 概念：一次等多个文件描述符
    - 3.4.2 为什么需要它：单 fd 接口的局限
    - 3.4.3 核心价值：把"等"和"拷"拆开
  - 3.5 异步 IO
    - 3.5.1 概念：只发起，不参与
    - 3.5.2 工作过程：aio_read + 内核全包
- 4. 同步 IO 与异步 IO
  - 4.1 同步 IO：只要参与 IO 过程就是同步
  - 4.2 为什么信号驱动 IO 属于同步 IO？
  - 4.3 为什么多路复用属于同步 IO？
  - 4.4 异步 IO 的定义：只发起，不参与

    - 4.5 同步与异步的区分标准：有没有参与 IO
- 5. 非阻塞 IO 实战
    - 5.1 把文件描述符设置为非阻塞的四种方式
        - 5.1.1 创建时指定：O_NONBLOCK / SOCK_NONBLOCK
        - 5.1.2 open 时指定：O_NONBLOCK
        - 5.1.3 recv / recvfrom 的 MSG_DONTWAIT
        - 5.1.4 fcntl 修改：最通用
    - 5.2 代码：从阻塞读到非阻塞读
        - 5.2.1 基础读取代码（阻塞版）
        - 5.2.2 SetNonBlock 函数
        - 5.2.3 设置非阻塞并测试
        - 5.2.4 完整代码
    - 5.3 errno 的区分
        - 5.3.1 EWOULDBLOCK / EAGAIN：数据没就绪，不是错误
        - 5.3.2 EINTR：被信号中断，也不是错误
        - 5.3.3 其他：真正的 read error
    - 5.4 非阻塞读的标准写法
- 6. 总结与下一步
    - 6.1 五种 IO 模型回顾
    - 6.2 重点：非阻塞 IO + 多路转接

1. IO 是什么?

1.1 IO 的本质：等 + 拷贝

如何进行高效的网络通信？就要对 IO 本身进行理解。
IO 是什么？Input/Output：外设 <-> 内存的相互拷贝，网络通信的本质是属于 IO 范畴的，实际上我们可以说 **IO = 等数据就绪 + 拷贝**。

> 在平时网络操作中，进行 read 时感觉比较慢，如果客户端不给你发数据，你调用 read，read 就是不返回，卡在那里。不管是 output 还是 input，其本质都等于等加拷贝。

1.2 IO 慢的根本原因：等的时间太多

依照上面的总结，IO 的本质是：**等 + 拷贝**。

拷贝效率的高低在系统层面和网络层面是固定的，完全取决于硬件。从内存拷贝到磁盘、从磁盘拷贝到内存、从内存拷贝到网卡、从网卡拷贝到内存，包括从内核拷贝到用户、从用户拷贝

到内核，整个过程很难在拷贝级别上做优化，只能减少拷贝的次数。如果拷贝次数固定，拷贝效率的高低跟硬件强相关。

所以软件层能优化的，只有优化"等"这一部分。

1.3 什么叫做高效的 IO?

高效的 IO 定义：单位时间之内能够进行更多次的数据交互。

所谓高效的 IO，在一秒以内，如果以前 98% 的时间都是在等，只要在一秒内能够让等待的时间降到 90%，其中 IO 的比重就增加了，效率也就变高了。

在单位时间之内，拷贝数据越多越好。单位时间之内要么在做等，要么在做拷贝。拷贝时间越长、拷贝的数据量越大，本质转化思路就是等的比重就减少。

低效的 IO 是大多数时候都在等的 IO。等的比重越低，IO 效率越高。

> 所以，如何设计高效的 IO？——减少"等"的占比。
> 只要能够想尽一切办法减少单位时间内 IO 中等的比重，那么 IO 效率就是最高的。

这就是接下来五种 IO 模型要解决的核心问题。

---

2. 讲一个故事，直观理解 IO

2.1 背景阐述

我们平常也都听过或者见过钓鱼吧，钓鱼过程简化为两步：第一步是等（等鱼咬钩），第二步是钓（把鱼从河里钓上来）。

钓鱼效率高的人：单位时间之内等的比重非常低。最高效的是这个人不等，一直在挥舞着鱼竿，鱼一直咬钩，跟流水线一样一直在做调度动作。

> 这实际上跟 IO 完全对应：**等 = 等数据就绪，钓 = 拷贝数据。**
> 那么同时可以比喻：鱼漂 = 文件描述符，鱼漂动了 = 数据就绪。

鱼漂的作用：一旦鱼钩扔到河面，鱼漂就立起来。当鱼咬钩时，鱼漂就会上下浮动。

鱼漂表明鱼就绪，鱼漂动的过程叫做条件就绪，对应 IO 当中数据就绪。
鱼漂就可以比作文件描述符。

2.2 五种钓鱼方式

2.2.1 张三 —— 阻塞 IO



张三把鱼钩往河里一扔，坐在马扎上，两个眼睛死死盯着鱼漂，鱼漂不动他不动的。大概等了 20 多分钟，鱼漂开始上下浮动，张三检测到后立马把鱼竿拎起来钓上第一条鱼。钓上鱼后继续挂饵料扔到河里观察鱼漂，重复动作。

特点：会一直关注鱼上钩，鱼一旦上钩鱼漂一动，立马识别并完成钓鱼。如果鱼漂不动，张三就一直盯着鱼漂，什么都不做。

2.2.2 李四 —— 非阻塞 IO



李四把鱼钩扔到河里后，拿眼睛扫一眼鱼漂，发现没有鱼咬钩，立马就不看鱼漂了。他转向其他地方，跟张三说话、刷抖音、看书。过一会儿再检测一次鱼漂，发现没有鱼咬钩就重复刚才的动作。他不会因为鱼漂没有鱼咬钩而卡住自己，在钓鱼过程中还做了其他事情。

特点：轮询式地关注鱼漂，不会因为鱼漂没有鱼咬钩而卡住自己。

2.2.3 王五 —— 信号驱动 IO



王五从口袋里摸出一只铃铛，挂在鱼竿的顶部。把鱼钩扔到河里后，手不拿鱼竿，把鱼竿直接插在岸边。王五坐在小马扎上翘着二郎腿看报纸，头也不抬。过了 20 多分钟，铃铛叮铃开始响了，王五头也不抬一把把鱼竿拽上来把鱼钓了上来。

特点：通过铃铛来通知自己。本质是**逆转了通知方式**——鱼咬钩了，鱼自己来通知王五。王五不再主动检测，而是让鱼来通知他。

2.2.4 赵六 —— 多路复用 / 多路转接



赵六开了一个小货车，拉了 100 根鱼竿。给每个鱼竿挂上诱饵，把 100 个鱼竿插在岸边。赵六在 100 个鱼竿旁边来回进行检测，突然发现有一个鱼竿有鱼咬钩，立马把这个鱼竿钓起来。钓完第一个后，紧接着第五个鱼竿又有鱼咬钩了，赵六立马再钓上来。

特点：同时监测多个鱼竿上的鱼漂事件。一次等 100 个鱼竿，等的时间上重叠了。

2.2.5 田七 —— 异步 IO



田七是一名成功的商人，不喜欢钓鱼，喜欢吃鱼。田七让司机小王去帮他钓鱼。田七给小王一套渔具、一个水桶和一部电话，告诉小王：把鱼钓满 20 条时，用电话通知他，田七过来接小王和鱼。田七自己则去公司处理紧急事情。

特点：田七自己不参与钓鱼，只发起钓鱼这件事，由小王替他完成整个钓鱼过程。

2.3 五种钓鱼方式的效率对比：赵六为什么最快?

假设他们一起去钓鱼。那么在河里有 104 个诱饵（张三 1 个、李四 1 个、王五 1 个、小王 1 个、赵六 100 个），每条鱼咬钩的概率相等。咬张三、李四、王五、小王的概率都是 104 分之一，而咬赵六的概率是 104 分之 100。

赵六鱼竿多，诱饵的概率高，单位时间之内任意一个鱼竿上鱼就绪的概率比其他人高，等的比重永远最低，所以钓鱼效率最高。

赵六这种同时检测多个鱼竿上是否有就绪事件的方式，本质可以在单位时间之内减少 IO 当中等的比重，这就是传说中的多路复用或多路转接。

---

3. 五种 IO 模型详解

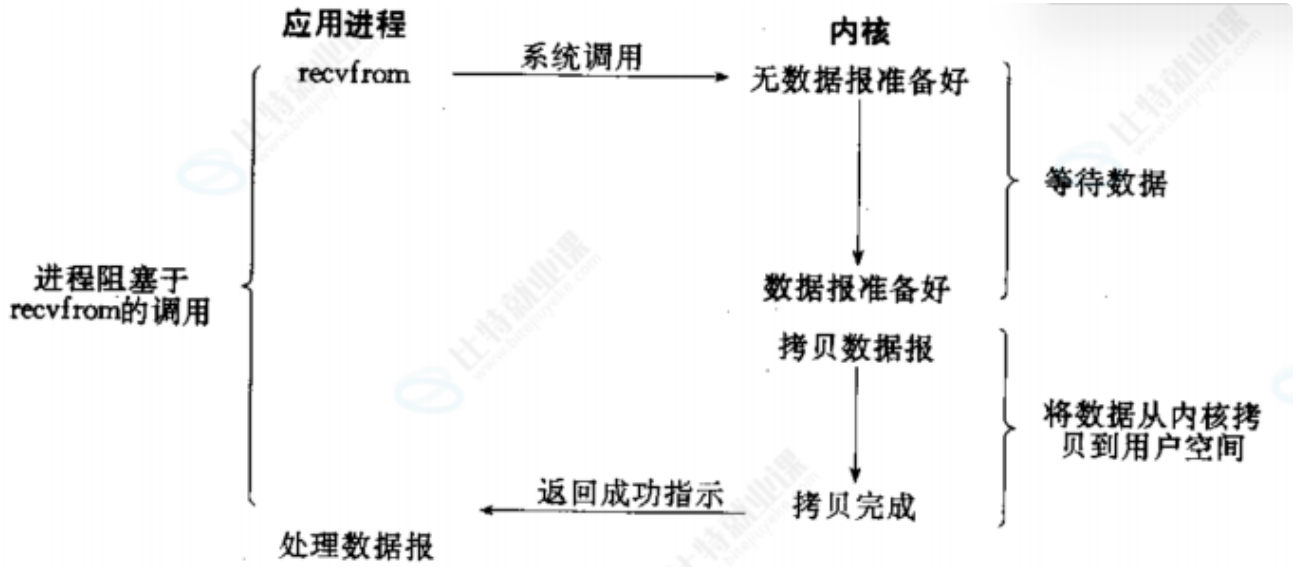
3.1 阻塞 IO

3.1.1 概念：最常见、最简单、最可靠

阻塞 IO 是从学习开始遇到的最常见的一种 IO 模型，也是真实应用场景中应用最广泛的 IO 之一。阻塞 IO 特别简单可靠，应用场景最多。

> 以前学过的 read、write、receive、send，包括读写文件、读写网络，所有 IO 全都是阻塞 IO。

3.1.2 工作过程：等 + 拷都在内核完成



阻塞 IO 是在内核将数据准备好之前。平时用到的 read、write、receive、send，不管读文件还是读网络，本质是把文件所对应的内核缓冲区里的数据拷贝到用户，或者从用户拷贝到内核。程序不直接跟硬件打交道，程序只跟操作系统打交道。

以读为例，在操作系统没有把数据准备好之前，系统调用（如 read）会一直阻塞。所有套接字默认都是阻塞方式。阻塞 IO 是最常见的 IO 模式，没有之一。

进程调用 recvfrom（UDP 的读接口），如果底层没有数据准备好，就要等数据就绪。一旦准备好了（网卡收到数据，把数据搬到内存里，搬到缓冲区里），再将数据从内核拷贝到用户空间，拷贝完成返回，然后处理数据包。

进程阻塞于 recvfrom 调用，recvfrom 做了两件工作：一个等，一个拷。

3.2 非阻塞 IO

3.2.1 概念：等的时候不会卡住

非阻塞 IO 在等的时候不会卡住。阻塞 IO 和非阻塞 IO 都是在等 IO 就绪，只是阻塞 IO 会卡住，非阻塞 IO 会立即返回，应用场景不一样。

3.2.2 工作过程：没就绪立即返回



如果内核没有把数据准备好，系统调用会直接返回（返回 EWOULDBLOCK 错误）。如果数据准备好了也会返回。

应用程序以非阻塞方式进行读取时，调用 recvfrom 进行系统调用，没有数据准备好会立即返回。在第一次和第二次之间可以做其他事情。

这个过程本质是通过轮询的方式检测数据有没有准备好。一旦某一次发现数据准备好了，拷贝数据到内核，拷贝完成返回。

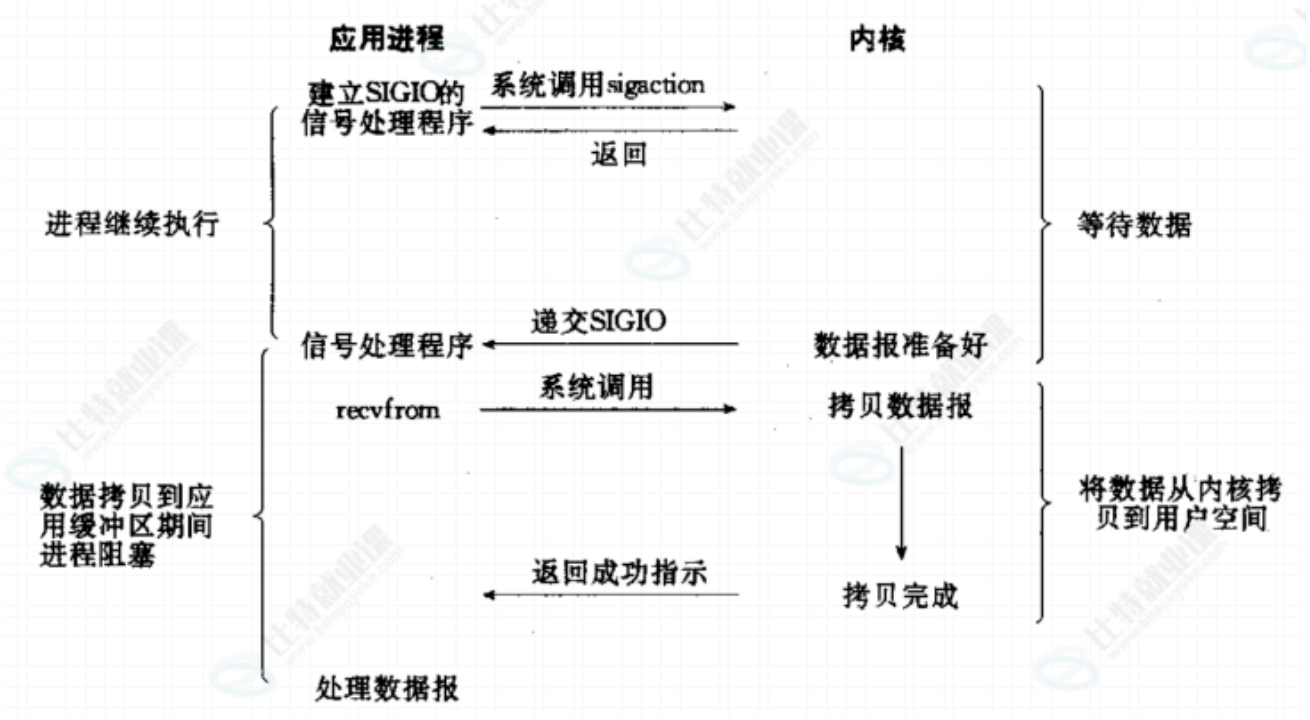
> 代价：轮询对 CPU 是一种浪费
> 非阻塞往往需要程序员自己以循环的方式尝试读写文件描述符，这个过程叫做轮询。**这种轮询过程对 CPU 来讲是一种比较大的浪费，一般只有特定场景下才会这样用。**

3.3 信号驱动 IO

3.3.1 概念：通过 SIGIO 信号通知

在 Linux 系统中，有一些信号叫做 SIGIO。当底层有数据就绪时，操作系统可以配置成给目标进程发信号。

3.3.2 工作过程：绑定 SIGIO + O_ASYNC



做法：对 SIGIO 信号进行捕捉处理，在信号捕捉处理函数里面调用 receive。在初始化时绑定 SIGIO，通过 fcntl 打开 O_ASYNC，把信号驱动 IO 开启。当底层有数据就绪时，操作系统会给进程发 SIGIO 信号，进程在信号捕捉函数中进行读取。

当内核把数据准备好的时候，内核可以通过 SIGIO 信号来通知应用程序进行 IO 操作。

具体做法：

- 进程调用 sigaction 或 signal 来设置对 SIGIO 的自定义处理（先捕捉这个信号）。
- 开启对 SIGIO 的自定义捕捉后，进程开始忙自己的事去了。但进程不能自己退，变相地他还在等数据。
- 突然有一个时刻数据准备好了，内核递交 SIGIO 信号告诉进程数据就绪。
- 进程才调用 recvfrom 进行读取。

信号驱动 IO 中，进程不需要等了，只需要拷贝就行了。但拷贝是进程自己亲自拷的，和阻塞 IO、非阻塞 IO 自己亲自拷是一模一样的。所以信号驱动 IO 本质上也属于同步 IO 范畴。

> 信号驱动 IO 虽然设计巧妙，但在高 IO 情况下信号会丢失（信号只有位图记录），所以出场率最低。

3.4 多路复用 / 多路转接

3.4.1 概念：一次等多个文件描述符

目前学到的 read、write 等各种系统调用函数，传参时只能传一个文件描述符，一次只能读一个。即使是非阻塞读写也是只能读一个。信号驱动也只能一次处理一个文件描述符。

而多路转接接口（如 select、epoll）一次可以帮助等待多个文件描述符，甚至可以向一个系统调用函数里直接传几百、上千个文件描述符。

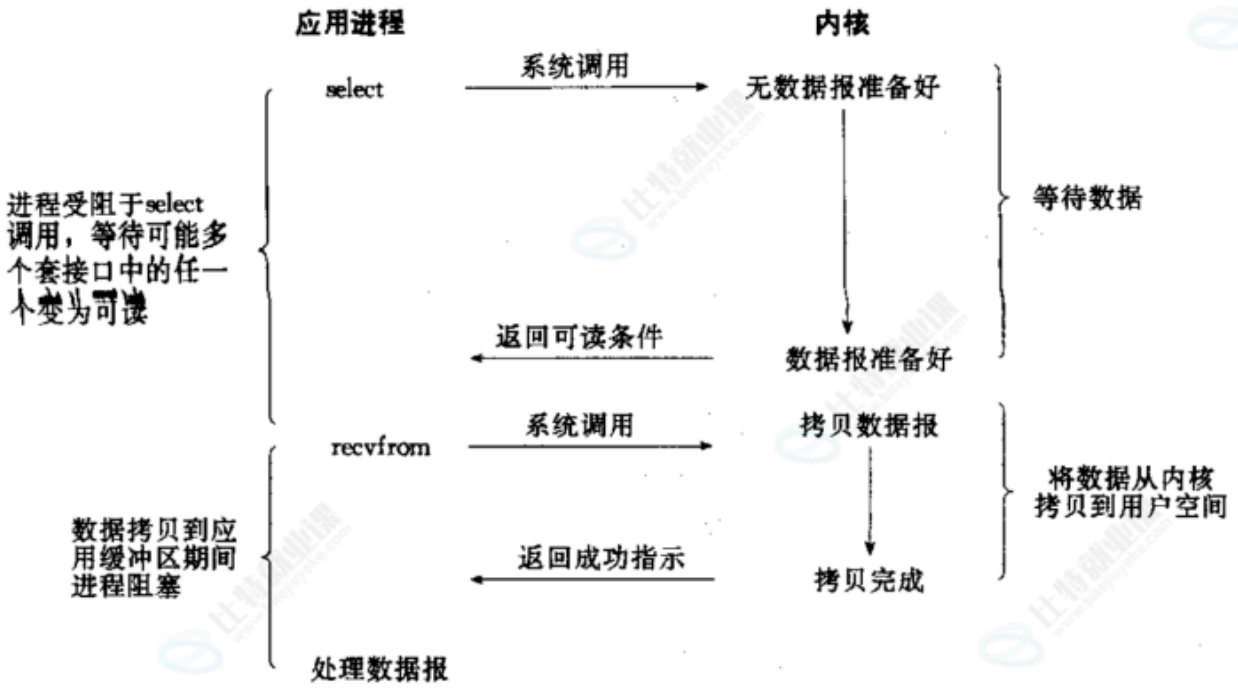
3.4.2 为什么需要它：单 fd 接口的局限

以前的 `recvfrom` / `read` 这类接口，一次只能处理一个 fd。它们内部做了两件事：等数据就绪 + 拷贝数据。因为一次只能拷一个 fd，所以一次也只能等一个 fd。

3.4.3 核心价值：把"等"和"拷"拆开

为了能够支持这种多路转接的等待方式，在 Linux 内核当中设计了一种新的系统调用。因为 IO 负责分为等和拷贝，所以使用特定的系统调用，比如 select、poll、未来的 epoll，让这样的系统调用只负责在 IO 过程当中的第一件事叫做等。由 select、poll 这样的系统调用只负责在 IO 当中的一件事情叫做等，而让 read、recvfrom 这样的接口只做另一件事情叫做拷。

> 更精细化的把接口拆分成一个叫做等，一个叫做拷。



拷贝函数已经定型了，而且拷贝永远都是一次只能拷贝一个文件描述符数据，一个个拷，不能一次拷贝数据拷贝多个文件描述符数据，因为不知道到底拷给谁。**拷贝数据必须明确文件描述符。**

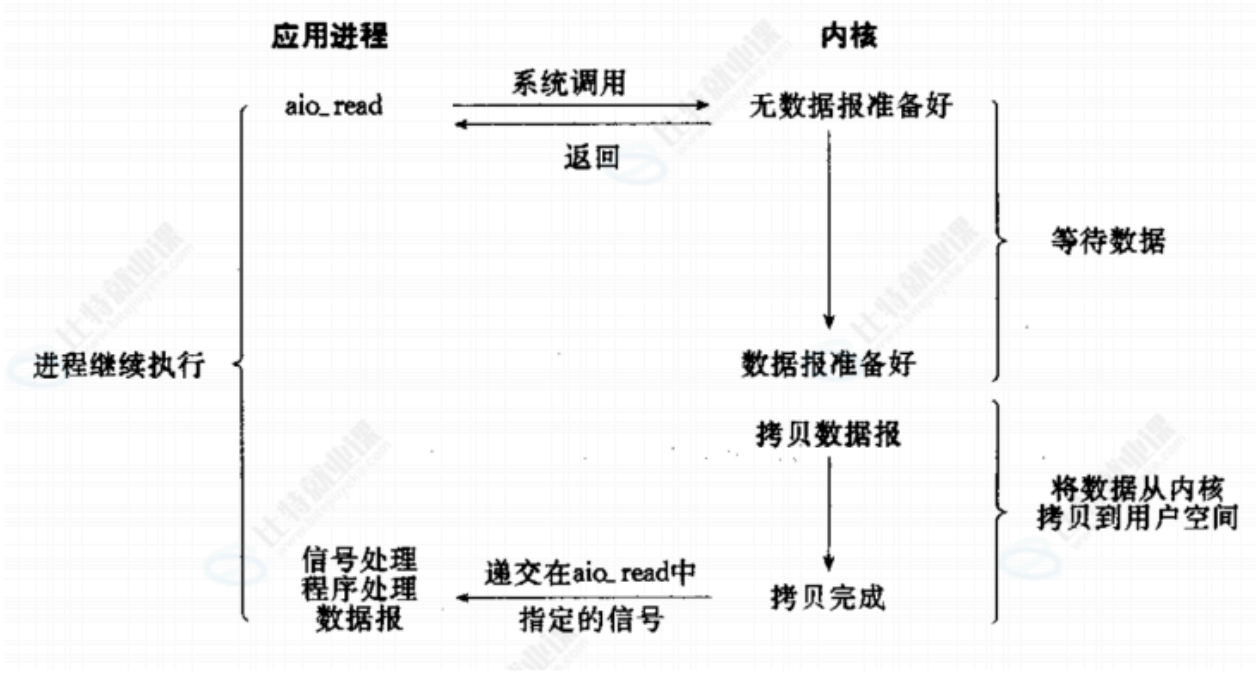
但是等的时候可以让 select 这一个系统调用一次设计的时候，让它可以同时帮我等多个文件描述符，一旦某一个就绪了，就通知上层，让上层调 recvfrom 进行拷。多路转接再具体一点，未来它会有新系统调用，**它等于把 IO 不光在理论上分成等和拷贝了，在操作上也给你分成等和拷贝。**

3.5 异步 IO

3.5.1 概念：只发起，不参与

异步 IO 在带来优势的同时也引入了新的缺点，在真实情况下出场率不高。尤其是在网络通信中存在多路转接技术以及协程技术后，异步 IO 变得比较鸡肋。

3.5.2 工作过程：aio_read + 内核全包



在 Linux 中，异步 IO 的原生接口叫做 aio_read。使用异步 IO 时，需要定义一个结构体 aiocb，设置文件描述符、缓冲区、读写、偏移量，以及通知方式和完成后的回调函数。

发起异步 IO 后，函数不会卡住，会继续往下走。由操作系统自动检测对应的文件描述符有没有数据，有数据时操作系统自动把数据拷贝到 buffer 里。

应用程序通过某些特殊的系统调用（如 aio_read）发起异步 IO。通常需要传递一些参数（结构体），设置要读哪个文件描述符、读多少数据、读成功后的回调函数等。

调用 aio_read 时，一调用立马返回，主线程继续执行，可以忙自己的事情。

进程不参与 IO，但要求内核帮进程进行 IO。等待数据的过程由操作系统自己做，当数据准备好了，将数据从内核拷贝到用户拷贝的过程也由操作系统做。

> 整个过程进程既没有调用 receive 去等，也没有调用 receive 去拷，根本没有参与 IO 的等和拷的任何一个过程，只是发起 IO。进程就是田七，内核就是小王。
> 当内核把数据拷贝完成后，通过 aio_read 指定的方式返回，告诉进程数据准备好了，该处理了。

---

4. 同步 IO 与异步 IO

4.1 同步 IO：只要参与 IO 过程就是同步

同步 IO：只要一个人或一个用户或一个系统调用参与了 IO 过程，就是同步 IO。

IO 分两步：一步是等，一步是调。要么两个都参与，要么可能只参与一部分，**只要参与就是同步**。

> 同步 IO 包含：阻塞 IO、非阻塞 IO、信号驱动 IO 和多路复用。

4.2 为什么信号驱动 IO 属于同步 IO?

信号产生的方式确实是异步的，只能说是通知机制是异步的。但王五有没有等？从物理上王五确实没等，但王五不敢走，走了铃铛响了也不知道。所以王五在逻辑上其实等了。

更重要的是，当铃铛响的时候，王五要亲手把数据从内核拷贝到自己的缓冲区。信号驱动 IO 中，一旦 IO 就绪，进程自己要调用 read 来拷贝数据。**说明王五至少参与了 IO 当中调的这一步。**

4.3 为什么多路复用属于同步 IO?

赵六只是等的鱼竿多，但每个鱼竿他都要自己过问。赵六在河边来回走的时候，本质也是在检测自己的鱼竿。他要参与等，也要参与调，只不过他一次等待多个鱼竿。

4.4 异步 IO 的定义：只发起，不参与

异步 IO：只负责发起 IO，不参与 IO 的任何细节。

IO 分两步：一步是等，一步是拷贝（或调）。在异步 IO 中，田七从逻辑上根本没有参与等，也没有参与调，他只是把钓鱼的事情发起了。等是小王等的，钓是小王钓的。田七从头到尾都在公司处理自己的事，完全不管钓鱼。**小王和田七两个人在逻辑上是完全异步的。**

进程从头到尾只负责发起 IO，等和拷都由内核完成——这就是异步 IO。

4.5 同步与异步的区分标准：有没有参与 IO

区分同步 IO 还是异步 IO，就看有没有参与 IO。

IO 等于等加拷贝。等的过程不参与，拷贝的过程也不参与，两步都不参与，就是异步的。**只要参与，哪怕只参与一部分，都是同步的。**

> 注意：这里的"同步"不是同步互斥
> 这里的同步 IO 与以前学的同步与互斥是完全不同的概念。
> **同步 IO 的核心点是：只要参与 IO 的过程就是同步的。异步 IO 不参与 IO 的过程，只发起 IO。**

---

5. 非阻塞 IO 实战

5.1 把文件描述符设置为非阻塞的四种方式

5.1.1 创建时指定：O_NONBLOCK / SOCK_NONBLOCK

在创建 fd 的系统调用中直接传入标志位。这是**最推荐**的方式。用 O_NONBLOCK、SOCK_NONBLOCK、EFD_NONBLOCK 等。

创建套接字时也可以创建 SOCK_NONBLOCK。

5.1.2 open 时指定：O_NONBLOCK

平时在对文件做操作时，当对文件系统层面上要对文件操作，也可以配置，因为文件也是跟网络一样，也是设备或者叫做文件。

在调用 open 时，有对应的 flag，打开文件的时候有读写、新建、追加等，还有一个标志叫做 O_NONBLOCK。如果以 O_NONBLOCK 的方式把文件打开，往后读这个文件描述符，它默认就是非阻塞，当调用 read 的时候，如果数据没有就绪，read 就会直接返回。

5.1.3 recv / recvfrom 的 MSG_DONTWAIT

```
recv(2)                                                    System Calls Manual

NAME
       recv, recvfrom, recvmsg - receive a message from a socket

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/socket.h>

       ssize_t recv(int sockfd, void buf[.len], size_t len,
                    int flags);
       ssize_t recvfrom(int sockfd, void buf[restrict .len], size_t len,
                        int flags,
                        struct sockaddr *_Nullable restrict src_addr,
                        socklen_t *_Nullable restrict addrlen);
       ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);
```

设置标志位

在网络里可以读数据，叫做 recvfrom 或 recv。不管是 recv 还是 recvfrom，它都有一个 flag 字段，叫做标志位。自 Linux2.2 内核之后，宏 MSG_DONTWAIT 的作用是开启 nonblocking operation，叫做非阻塞模式。

如果将来要进行 recvfrom，直接可以给这个标志位上面设置上 MSG_DONTWAIT，它在读的时候就以非阻塞的方式帮你读了。

> 写也一样，不管 UDP、TCP 都一样，一切皆文件，只要能够把一种设置规则弄懂，其他的都一样。

5.1.4 fcntl 修改：最通用

虽然把文件描述符设为非阻塞，包括把某些接口让它以非阻塞方式工作，方式千差万别，但 Linux 下一切皆文件，文件对应文件描述符，不管是普通文件还是 TCP、UDP，只要有文件描述符在内核当中，必然有 struct file 结构体，只要有 struct file 这个结构体，它必然有它自己对应的打开文件的 flag 标志位。

我们可以直接基于文件描述符设置，也就是 fcntl 加 O_NONBLOCK。

可以通过修改一个文件描述符的打开标志位当中，它所对应的标志位给它添加上非阻塞，把文件描述符自己设置为非阻塞。往后不管调什么 read、write、send、receive 这样的接口，文件描述符工作时都会处于非阻塞方式工作，即便这个 flag 设为零，它都是非阻塞的。

```
fcntl(2)

NAME
       fcntl - manipulate file descriptor

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <fcntl.h>

       int fcntl(int fd, int op, ... /* arg */ );
```

所以为了将一个文件描述符设置为非阻塞，**最推荐的做法就是使用系统调用 fcntl**。它的作用是可以去更改一个文件对应的打开标志类属性，头文件是 fcntl.h 和 unistd.h。fcntl 函数第一个参数是要设置或获取的文件描述符，第二个参数是要做什么命令，第三个是可变参数。

常见的有几种功能：复制一个文件描述符、获得一个文件的什么标志、获取一个文件描述符所对应的标记 flag、获取所有权、获取记录锁等。

> **最关注的是获取文件的 flag，即 F_GETFL。** 任何一个文件描述符在它的 struct file 结构体里包含一个 int flag 标志位，要把这个文件的 flag 标志位拿到。还可以设置，即 F_SETFL。

实现一个文件描述符变成非阻塞可以写个函数：调用 fcntl 获取一个文件描述符的标志位，获取成功保存下来，获取成果小于零则失败。一旦获取成功了，再调 fcntl 对指定的文件描述符设置标志位，在原来标志位的基础之上新增一个 O_NONBLOCK，此时 fd 文件描述符就被设置为非阻塞了。

5.2 代码：从阻塞读到非阻塞读

5.2.1 基础读取代码（阻塞版）

写一个从标准输入读取数据的代码。定义一个 int，大小 1024，在 read 的时候，0 就是标准输入，从标准输入里读，读到应用 buffer 里，读 size of buffer 减一。read 的返回值 n 如果是大于零，证明读成功了，把 buffer 中 n 的位置设为 0，然后输出 buffer。如果读取小于等于零，先简单处理。

编译通过之后运行，默认没有输入时，它就卡住了。输入 aa，它就回显 aa，输入 bb，它就回显 bb。其中有个问题，多了个空行，主要原因是因为当输入的时候，还按了一个回车，调用 read 从终端上输入读的时候，除了读到数据，最后把 `\n` 也读到了，在打印的时候字符串本来就有一个 `\n`，再加上一个换行，所以多了一个空行。可以给减个一解决。

> 当不往键盘里写的时候，这个代码就是阻塞的，会卡在 read 这里等去进行读取。

5.2.2 SetNonBlock 函数

```
#include <iostream>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

// 将IO设计为非阻塞的
void SetNonBlock(int fd)
{
    int fl = fcntl(fd, F_GETFL); // 将文件描述符设计为非阻塞
    if (fl < 0)
    {
        std::cerr << "fcntl error" << std::endl;
        return;
    }
    fcntl(fd, F_SETFL, fl | O_NONBLOCK);
}
```

5.2.3 设置非阻塞并测试

要把标准输入 0 设为非阻塞。使用 fcntl，先 fd，用 F_GETFL 获取当前文件描述符所对应的标记位。一般接口调用成功时返回 int，成功时获取到了对应的标志位，失败时返回 -1。如果 fl 小于 0，证明获取失败了，直接输出错误。如果成功了，再调 fcntl 对当前的文件描述符进行设置，设置标志位不能改老的标准，所以要重新获取一次，在老的标志位基础之上新增一个 O_NONBLOCK。将文件描述符设为非阻塞，告诉操作系统，操作系统看着文件描述符对应文件将来读写的时候必须是非阻塞的。

设置零号文件描述符为非阻塞后，尝试读。为了看到现象，让它慢一点，要不然会刷屏。零已经被设为非阻塞了，在读取时，它不会发现没有就绪，会立即返回。数据成功了也会立即返回，返回反正无内耗，立马返回。要么成功，要么就是失败。

编译运行，当调 read 的时候，在键盘上不做任何输入，read 就立马返回，只不过它目前是以出错形式返回的。输入 a、b、c、d 的时候，这些字符只是进入了终端输入缓冲区，并没有提交给进程。只有按下回车，终端才会把这些数据一次性提交给进程，此时 read 才认为数据就绪。当把 abcd 输入到输入缓冲区，按一下回车数据提交给进程之后，进程把数据读到了，read 也会返回，读到对应的数据。

基于这种非阻塞，有数据就读，没数据就立即返回。在这个过程中可以做其他事情，比如问问张三钓多少条鱼了，从口袋里摸出来一个手机，刷刷抖音，看看会儿书，访问数据库，打印日志，访问 redis，做各种其他操作。**这就叫做非阻塞读。**

5.2.4 完整代码

```
#include <iostream>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

// 将IO设计为非阻塞的
void SetNonBlock(int fd)
{
    int fl = fcntl(fd, F_GETFL);
    if (fl < 0)
    {
        std::cerr << "fcntl error" << std::endl;
        return;
    }
    fcntl(fd, F_SETFL, fl | O_NONBLOCK);
}

int main()
{
    char inbuffer[1024];
    SetNonBlock(0);
    while (true)
    {
        ssize_t n = read(0, inbuffer, sizeof(inbuffer) - 1);
        if (n > 0)
        {
            inbuffer[n] = 0;
            std::cout << "echo# " << inbuffer << std::endl;
        }
        else if (n == 0)
        {
```

```
            std::cout << "end of file" << std::endl;
            break;
        }
        else
        {
            // read error || 如果一个文件描述符，读取的时候如果没有就绪，也是以出错
的形式返回的
            // 但是没有就绪不是错误！你怎么区分它究竟是错误还是没有就绪？
            if (errno == EWOULDBLOCK || errno == EAGAIN)
            {
                // 如果我们的错误码等于11，就不是错误
                // 进程就可以做自己的事情了
                std::cout << "data is not ready!" << std::endl; // 非阻塞轮询
            }
            else if (errno == EINTR) // 中断
            {
                // 被信号中断了，但是不属于错误
                continue;
            }
            else
            {
                std::cerr << "error..." << errno << std::endl;
                std::cerr << "error..." << strerror(errno) << std::endl;
                // 11：资源临时不可用，也就是你的资源没有就绪
            }
        }
        sleep(1);
    }
}
```

5.3 errno 的区分

5.3.1 EWOULDBLOCK / EAGAIN：数据没就绪，不是错误

当去读 0，本质是检测 0 号文件描述符和标准输入有没有数据，有数据就读到了，返回值大于零。如果返回值等于零，它代表的是读到文件结尾，类似于文件结尾，所以可以 break。

在 Linux 当中，从键盘里输入 Ctrl+D 告诉对应的进程已经读完了，Ctrl+D 就代表的是读到文件结尾，对于它来讲，相当于读到 0，返回值为 0，读到文件结尾。

当返回值小于零时，不仅仅是读取失败了，当在读时，0 号文件描述符如果没就绪，它也是以出错形式返回的。如果一个文件描述符读取的时候，如果没有就绪，也是以出错形式返回的。

> 读出错和数据没就绪，它都会以出错形式返回，但是数据没就绪不是错误。

```
#define ENOTEMPTY   39  /* Directory not empty */
#define ELOOP       40  /* Too many symbolic links encountered */
#define EWOULDBLOCK EAGAIN  /* Operation would block */
#define ENOMSG      42  /* No message of desired type */
#define EIDRM       43  /* Identifier removed */
#define ECHRNG      44  /* Channel number out of range */
#define EL2NSYNC    45  /* Level 2 not synchronized */
#define EL3HLT      46  /* Level 3 halted */
#define EL3RST      47  /* Level 3 reset */
#define ELNRNG      48  /* Link number out of range */
#define EUNATCH     49  /* Protocol driver not attached */
#define ENOCSI      50  /* No CSI structure available */
#define EL2HLT      51  /* Level 2 halted */
#define EBADE       52  /* Invalid exchange */
#define EBADR       53  /* Invalid request descriptor */
#define EXFULL      54  /* Exchange full */
#define ENOANO      55  /* No anode */
#define EBADRQC     56  /* Invalid request code */
#define EBADSLT     57  /* Invalid slot */
```

操作将会被阻塞

现在的问题就过渡到了怎么区分具体是错误还是数据未就绪。在非阻塞情况下必须对出错做区分了。以前是阻塞调用，阻塞调用只要小于零，它必然出错，不存在这种情况。今天是非阻塞了，多了一种情况了，所以必须区分到底是真的出错了，还是底层数据没就绪。

在 C 语言中，通常表明一个函数本身在调用的时候，在功能上有没有调用成功，可以通过返回值看到，但是这个函数在调用期间有没有出错，在 C 语言中会提供一个隐性的全局变量叫做 errno。Errno 通常会用来表示错误码，它可以告诉你调用或者是出错的原因是什么。

当调用 read 的时候，在 read 本身也是 C 语言的封装，它底层也会当调用成功和失败的时候，会返回值表明它失败了。read 调用失败，它会设置 errno。

通过运行可以看到当前对应的 errno 对应的错误码叫做 11。通过 strerror 接口可以把错误码转化成错误码描述，strerror 是一个把错误码转化成错误码描述的一个接口。运行后显示错误码是 11，原因是资源临时不可用，说白了就是文件描述符没就绪。

```
#define EPERM        1  /* Operation not permitted */
#define ENOENT       2  /* No such file or directory */
#define ESRCH        3  /* No such process */
#define EINTR        4  /* Interrupted system call */
#define EIO      5  /* I/O error */
#define ENXIO        6  /* No such device or address */
#define E2BIG        7  /* Argument list too long */
#define ENOEXEC      8  /* Exec format error */
#define EBADF        9  /* Bad file number */
#define ECHILD      10  /* No child processes */
#define EAGAIN      11  /* Try again */
#define ENOMEM      12  /* Out of memory */
#define EACCES      13  /* Permission denied */
#define EFAULT      14  /* Bad address */
#define ENOTBLK     15  /* Block device required */
#define EBUSY       16  /* Device or resource busy */
#define EEXIST      17  /* File exists */
#define EXDEV       18  /* Cross-device link */
#define ENODEV      19  /* No such device */
#define ENOTDIR     20  /* Not a directory */
#define EISDIR      21  /* Is a directory */
#define EINVAL      22  /* Invalid argument */
#define ENFILE      23  /* File table overflow */
#define EMFILE      24  /* Too many open files */
#define ENOTTY      25  /* Not a typewriter */
```

所以真正要写对应的代码，应该是如果 errno 等于 11，11 叫做 EWOULDBLOCK，或者叫做 EAGAIN，两个写一个都行。EAGAIN 意思就是 try again，它的意思就是 11。如果错误码等于 11，那么就不是报错，叫做 data is not ready，数据没就绪。

数据没准备好，输入一些东西回车数据就准备好，就打印了，打印完之后读取完，下一次再读数据，又没准备好，这叫做非阻塞轮询。调用 EWOULDBLOCK，没准备好，下次调好了，拷贝完成。

除了它之外 else，只有除了它之外，才叫做读取失败，表明读取失败了，叫做 read error。

5.3.2 EINTR：被信号中断，也不是错误

信号可以中断一个进程的阻塞过程。一个进程如果被阻塞了，它是能够被信号唤醒的，因为只有醒来了，才会进行信号捕捉和处理，只有信号处理了，才会终止。信号有可能会中断一个进程的阻塞过程。

如果写了一个进程，sleep 上 100 秒，当进程 sleep100 秒的时候，后面直接把这个进程 ctrl+c 掉或者 kill-9 杀掉。杀进程的时候，是向进程 PCB 里写对应的位图，但进程在休眠，给

它写了信号，它也处理不了。所以发信号的过程，不光发信号了，还要把进程唤醒。只有当它唤醒的时候，它才会执行自己的信号处理方法，才会退出。**信号其实也有可能会影响阻塞状态，进程可能会因为收到一种信号而把进程提前唤醒。**

在写非阻塞的时候，虽然非阻塞这种情况去读取的时候概率特别低，但也确实会存在一些情况。当读取某一个文件描述符的时候，会因为进程收到了某一个信号而导致 read 提前返回，可能并不是因为检测到非阻塞了，可能就是读取还没到检测条件那一步，发现时间没就绪，然后可能收到信号就直接返回了。这也是为了更好的适配阻塞调用，因为当阻塞的时候，突然被信号中断唤醒了，可能不是致命的，对信号的处理动作可能是某种忽略的，或者是自定义什么都不做的动作，但是它就是把我从休眠状态直接唤醒了。信号可能会让你提前唤醒。

在 Linux 当中所罗列的错误信息中，有一个 EINTR，表示的是一个函数调用被中断了，一般代表的是被信号中断了。所以稳妥起见，还要再加一个判断，如果 errno 等于 EINTR，EINTR 的意思就是 interrupt，表示的是当前所对应的 read 被信号中断了。**被信号中断或者数据没就绪，直接 continue。只要非阻塞了，或者 IO 被信号中断了，直接考虑要继续读，因为它不属于错误。除了这两种情况之外，再进行 else 操作处理。**

5.3.3 其他：真正的 read error

除了 EWOULDBLOCK、EAGAIN 和 EINTR 之外，其他 errno 才代表真正的读取失败。

5.4 非阻塞读的标准写法

非阻塞 IO 当中一种完善的、较为完善的一般写法：

```
if (errno == EWOULDBLOCK || errno == EAGAIN || errno == EINTR)
{
    continue;
}
else
{
    // 真正的错误
}
```

在大部分情况下，它对应的都是 continue，一般都是没有就绪的。

当在进行非阻塞读取的时候，以 read 为例，将来也可以换成其他接口。非阻塞读跟正常读是一模一样，有数据就读，没数据无非就做更多事情了。

---

6. 总结与下一步

6.1 五种 IO 模型回顾

五种 IO 模型包括**阻塞 IO、非阻塞 IO、信号驱动 IO、多路复用和异步 IO**。

其中信号驱动和异步 IO 不作为重点来说，重点是非阻塞 IO 和多路转接。信号驱动基本不用，一般有可能要用的话，到时候再看就行了。

6.2 重点：非阻塞 IO + 多路转接

- **非阻塞 IO**：等的时候不卡住，可以去做其他事
- **多路转接**：一次等多个文件描述符，把"等"和"拷"从操作上拆开

这两个组合起来，就是高性能网络服务器的核心。
下一篇预告：select / epoll 多路转接。